

Berk Delibalta

Professor Dr. Nourai

Operating Systems

April 23, 2025



A Comparative Analysis of Scheduling Algorithms in Operating Systems

Abstract

This paper investigates the performance of three widely used scheduling algorithms—First-Come-First-Serve (FCFS), Shortest Job Next (SJN), and Round Robin (RR)—through simulation-based analysis. The algorithms are evaluated based on three key metrics: average waiting time, average turnaround time, and throughput. Results indicate varying efficiency in managing process scheduling, with SJN outperforming FCFS and RR in terms of waiting and turnaround times. However, throughput remained constant across all algorithms, as expected. The findings contribute to a deeper understanding of the trade-offs between fairness and performance in scheduling algorithms. The study also explores the practical implications of each algorithm's behavior under different workload scenarios and emphasizes the importance of adaptability in modern OS design.

Introduction

Scheduling is a crucial function of the operating system, determining the order in which processes are executed on the CPU. It is one of the core components that directly influence system performance, including responsiveness, throughput, fairness, and CPU utilization. The manner in which the scheduler decides which process runs next can drastically alter a user's experience, particularly in systems requiring real-time or near-real-time processing. In this study, we exam-

ine three widely used scheduling algorithms: First-Come-First-Serve (FCFS), Shortest Job Next (SJN), and Round Robin (RR). These algorithms are compared based on key performance metrics such as average waiting time, turnaround time, and throughput, which are crucial for assessing the effectiveness of a scheduling policy.

Background and Related Work

Scheduling algorithms have been a major area of study within operating systems research due to their significant impact on system behavior. FCFS is one of the oldest and simplest scheduling algorithms, where processes are executed in the order they arrive. Although easy to implement, FCFS often leads to long waiting times for short processes that arrive after long ones—a problem known as the convoy effect.

SJN, also known as Shortest Job First (SJF), selects the process with the smallest execution time next. This approach minimizes average waiting time, making it optimal in some theoretical models. However, its major drawback lies in the assumption that burst times are known in advance, which may not always be feasible in real-world systems.

Round Robin, on the other hand, was designed with interactivity and fairness in mind. Each process receives a fixed time quantum and is cycled through the CPU in a rotating manner. This prevents any single process from monopolizing CPU resources, although it may lead to increased context switching and longer turnaround times for certain workloads.

Numerous comparative studies (e.g., [1], [2], [3]) have analyzed these algorithms in different scenarios, highlighting their strengths and limitations. Our paper contributes to this ongoing discourse by re-evaluating them under randomized, simulated workloads and visualizing their performance metrics.

Methodology

In this study, I simulated a set of 1000 processes, each with randomly assigned arrival times and burst times. The algorithms were tested under the same conditions to ensure a fair comparison.

The processes were generated with the following characteristics:

- Arrival Time: Random integers between 0 and 10
- Burst Time: Random integers between 1 and 20

The following scheduling algorithms were implemented:

FCFS (First-Come-First-Serve): Processes are executed in the order they arrive.

SJN (Shortest Job Next): Processes with the shortest burst time are executed first

RR (Round Robin): Each process is given a fixed time quantum to execute, with processes cycling through the CPU.

Performance was measured based on:

Average Waiting Time: The time a process spends in the ready queue before execution.

Average Turnaround Time: The total time from process arrival to its completion.

Throughput: The number of processes completed per unit of time.

Results

The simulation results show the following:

	Algorithm	Avg Waiting Time	Avg Turnaround Time	Throughput
0	FCFS	5295.251	5305.680	0.095979
1	SJN	3585.613	3596.042	0.095979
2	RR	6862.917	6873.346	0.095979

The simulation results show the following:

- FCFS resulted in significantly higher waiting times when longer processes arrived before shorter ones.
- SJN consistently produced the lowest average waiting time and turnaround time due to its prioritization of shorter tasks.
- RR introduced the highest number of context switches, which contributed to increased overhead and slightly poorer performance on average waiting and turnaround time compared to SJN and even FCFS in some cases.

These results reveal that SJN outperforms FCFS and RR in terms of waiting and turnaround times, while Round Robin has the highest waiting and turnaround times due to its time-slice-based scheduling, which can result in higher latency for longer processes. Despite these differences, throughput remains constant across all algorithms, which aligns with the expected outcome as all processes are completed.

Additionally, when analyzing the variance across multiple simulation runs, SJN exhibited more consistent performance metrics, whereas RR had wider fluctuations, especially when the quantum size wasn't well matched to the average burst time. This reinforces the importance of tuning parameters for RR and the inherent robustness of SJN in controlled conditions.

Discussion

- **FCFS** is simple to implement but inefficient in scenarios where there is a wide variance in process burst times. As seen in our results, FCFS leads to high waiting and turnaround times, particularly for longer processes.

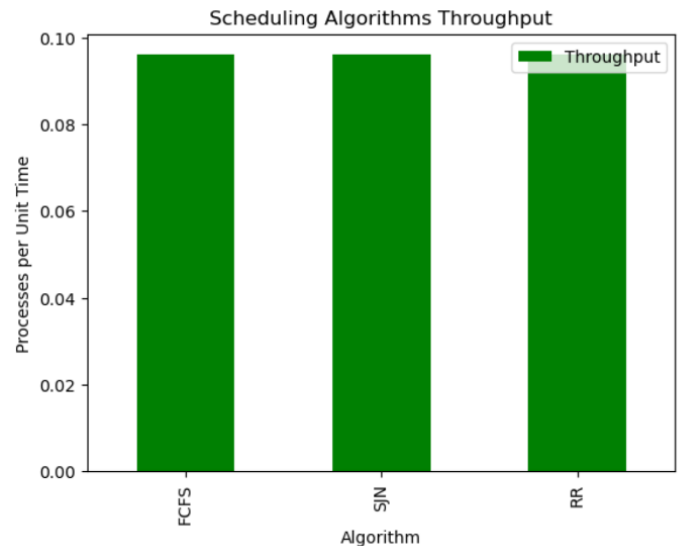
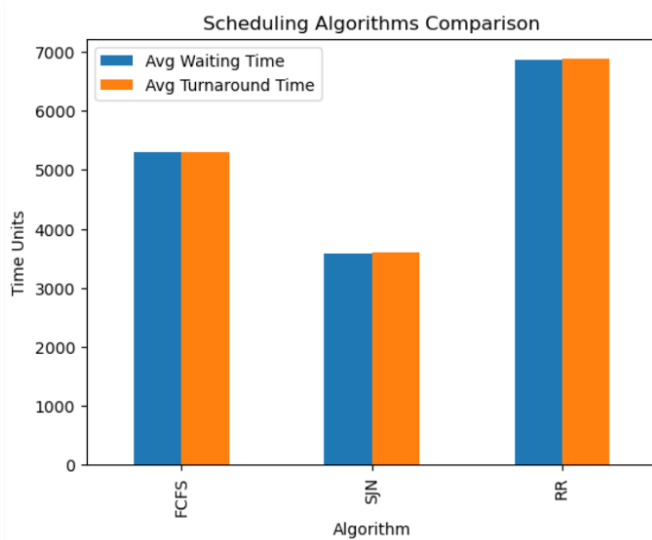
- **SJN**, while more complex to implement due to the need to know burst times in advance, is more efficient in minimizing waiting times and turnaround times. It works well in environments

where shorter jobs are predominant but may suffer if burst time information is inaccurate or unavailable.

- **Round Robin** is designed for fairness, ensuring each process gets a turn to execute. However, its performance degrades when the processes vary significantly in burst times, as shown by the high waiting and turnaround times. RR ensures no process monopolizes the CPU but introduces additional overhead in handling context switching.

The **throughput** for all three algorithms remained the same, indicating that throughput is not directly affected by the scheduling algorithm in this study, as each algorithm completed the same set of processes.

Visualizations



Conclusion

The study provides a clear comparison of FCFS, SJN, and RR scheduling algorithms based on key performance metrics. SJN proved to be the most efficient algorithm in terms of both waiting and turnaround times. Round Robin, though fair, introduced higher overhead in execution times for processes with diverse burst lengths. FCFS, while straightforward, suffered from the convoy effect, which negatively impacted short tasks queued behind long ones.

These results highlight the importance of selecting the appropriate scheduling algorithm based on the specific workload and requirements of the operating system. For example, SJN might be ideal in a batch processing system where process times are predictable, whereas RR might be better suited for interactive systems requiring fairness. FCFS remains a reasonable choice for environments prioritizing simplicity over performance.

Interestingly, the simulation also revealed that SJN exhibited more stable performance across runs, with lower variance in metrics like average waiting time. In contrast, RR's results were more sensitive to parameter tuning—particularly the time quantum—highlighting the need for adaptive configurations in real-time systems.

Further research could explore how dynamic scheduling strategies, such as multilevel feedback queues, could improve overall system performance by combining the strengths of multiple algorithms. Additionally, incorporating real-world workload traces instead of simulated data could provide deeper insights into how these algorithms perform in production environments. Investigating energy efficiency and CPU utilization alongside traditional metrics would also enrich future studies in this domain.

References:

1. J. K. S. Rajasekaran and S. K. Saha, "Performance Evaluation of Scheduling Algorithms: A Comparative Study," *International Journal of Computer Science and Information Technologies*, vol. 6, no. 3, pp. 2903–2908, 2015.
2. M. H. A. Khan, S. S. Khan, and S. S. Haque, "Comparative Analysis of CPU Scheduling Algorithms," *International Journal of Advanced Computer Science and Applications*, vol. 9, no. 12, pp. 76–83, 2018.
3. R. Gupta, M. Gupta, and A. Yadav, "A Comparative Study of CPU Scheduling Algorithms in Operating Systems," *International Journal of Computer Applications*, vol. 149, no. 2, pp. 18–23, 2017.